



ChurnGuard

Predict SaaS customer churn before they leave

Part 1 of 2 — **Product & Features Walkthrough**

Every screen, KPI and the explainable AI risk scorer — explained for interview prep.

Next.js 16

React 19

PostgreSQL + Prisma

NextAuth

Recharts

Claude / Gemini AI

Prepared 14 July 2026 · Author: Bhanu Prakash

o • What ChurnGuard is

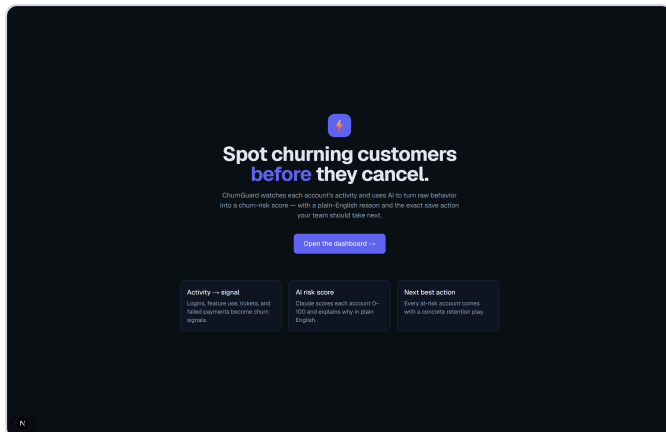
A full-stack customer-success console built on one principle: **churn risk is explainable, not a mystery**. Raw product-usage events are distilled into deterministic churn signals, an AI writes a plain-English risk rationale and a recommended action, and every re-score is snapshotted so you can watch a customer's risk move over time. The goal: spot the accounts about to leave *before* they do.

Interviewer one-liner: "It's a Next.js 16 app-router project where server components read PostgreSQL through Prisma; raw activity events are reduced to deterministic churn features, and an **LLM-as-judge** (Claude forced tool-use → Gemini structured JSON → a transparent heuristic) writes an explainable 0–100 risk score, band and recommended action; every score is upserted and appended to a history table that powers a live risk-trend chart."

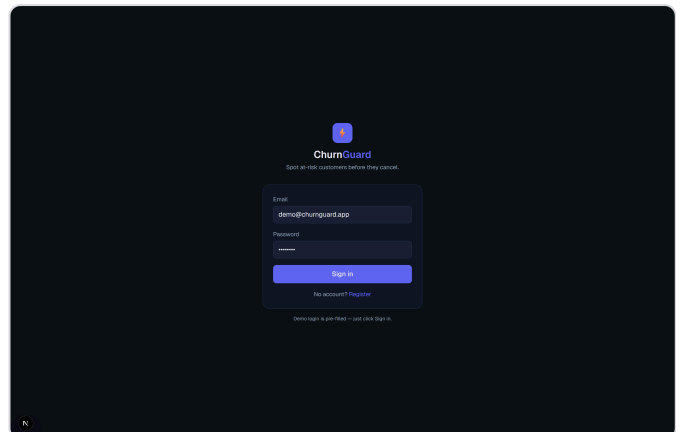
LAYER	CHOICE
Framework / UI	Next.js 16 (App Router, server components) · React 19 · TypeScript · Tailwind 4
Data	PostgreSQL + Prisma ORM (singleton client)
Auth	NextAuth (Credentials, JWT sessions) · bcrypt password hashing
Charts	Recharts (risk donut, revenue-by-band bars, risk-trend line)
AI scorer	Anthropic Claude (tool-use) → Google Gemini (REST) → deterministic heuristic

1 • Authentication

Email/password login via NextAuth with JWT sessions; passwords are bcrypt-hashed. The combined login/register page pre-fills the demo operator credentials; every protected page redirects to `/login` when there is no session.



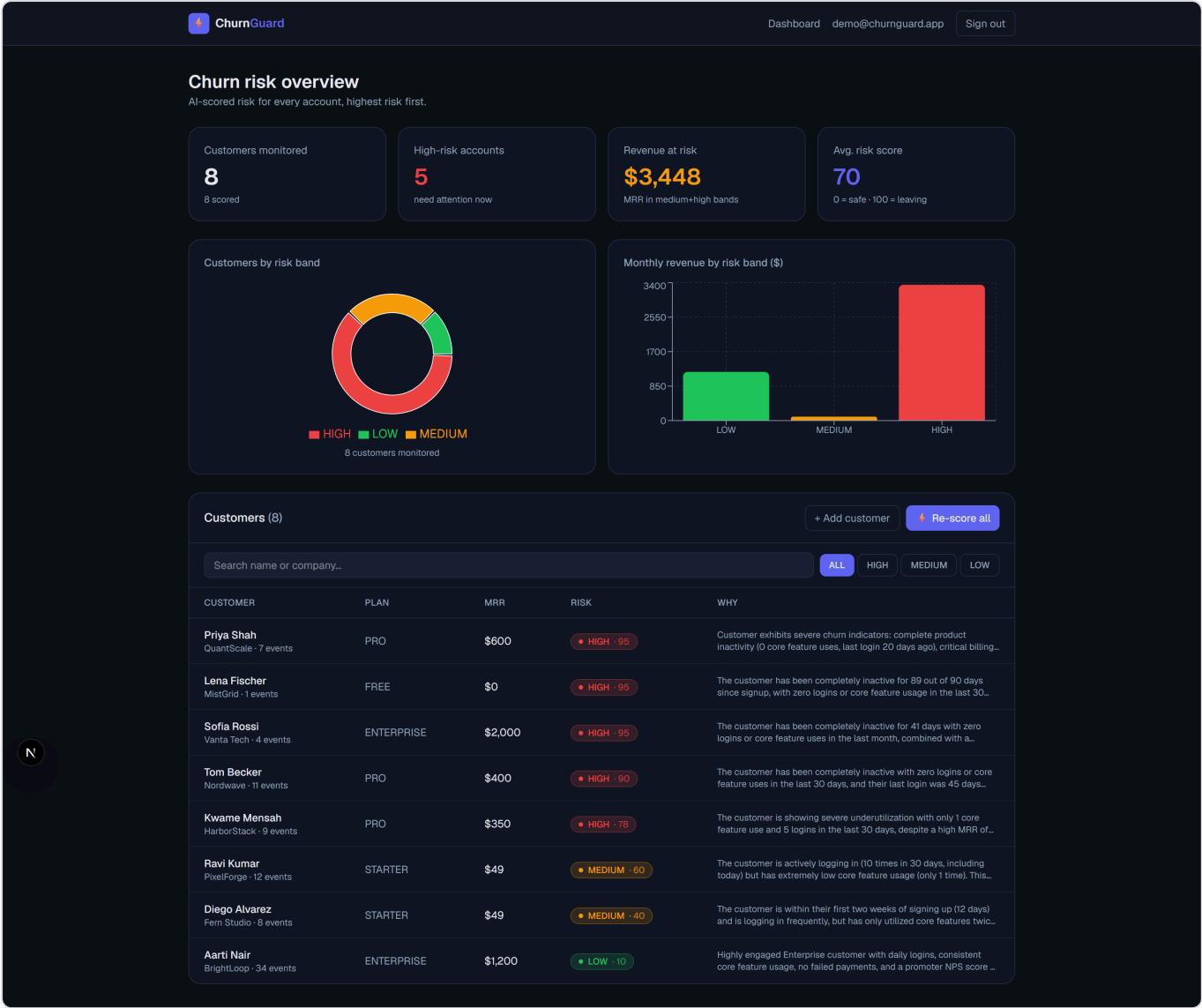
Marketing landing page (redirects to the dashboard if already signed in)



Combined login / register — demo creds pre-filled:
demo@churnguard.app / demo1234

2 · Dashboard — "Churn risk overview"

The whole book of business at a glance, sorted so the accounts most likely to leave are on top.



Dashboard: four stat cards, the "Customers by risk band" donut, the "Monthly revenue by risk band" bar chart, and the AI-annotated customer table (highest-risk-first) with search, band filter, "Add customer" and "Re-score all"

The four stat cards

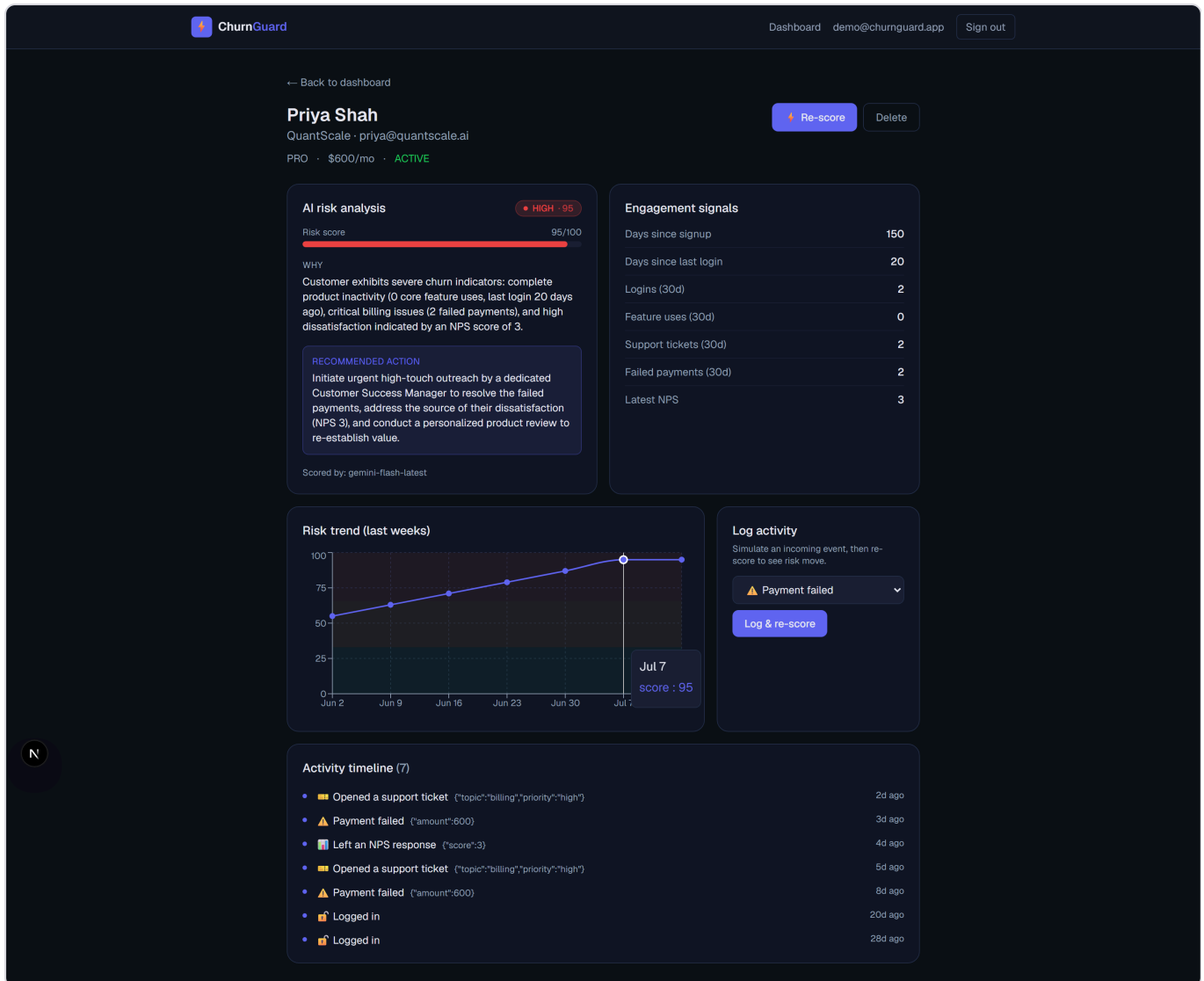
CARD	VALUE	MEANING
Customers monitored	8	Total active accounts being tracked
High-risk accounts	5	Customers currently in the HIGH band (score ≥ 66)
Revenue at risk	\$3,448	MRR sitting in the medium + high risk bands — the money in jeopardy
Avg. risk score	70	Mean score across all monitored customers (0–100)

The panels

- **Customers by risk band** — a donut splitting the book into HIGH / MEDIUM / LOW so you see the shape of the portfolio at a glance.
- **Monthly revenue by risk band** — a bar chart of MRR grouped by band; it makes "revenue at risk" concrete by showing exactly where the recurring dollars sit.
- **Customer table** — every account sorted highest-risk-first, each row carrying a colour-coded band badge, the numeric score, MRR, event count, and an AI "why" reason. Above it: a search box, a band filter, an **Add customer** action, and a **Re-score all** button that re-runs the scorer across the whole book.

3 • The Customer detail page

One account, fully explained — and a live loop where you log an event and watch the risk move.



Customer detail — Priya Shah, QuantScale (PRO, \$600/mo): AI risk analysis, engagement signals, the risk-trend chart, the log-activity form, and the activity timeline

- **AI risk analysis** — the headline panel: a score out of 100 with its band (here 95/100 HIGH), a **"Why"** paragraph explaining the drivers in plain English, a **"Recommended action"** for the CS team, and a **"Scored by"** line naming the engine that produced it (e.g. `gemini-flash-latest`).
- **Engagement signals** — the deterministic feature panel behind the score: days since signup, days since last login, logins in the last 30 days, feature uses, support tickets, failed payments, and the latest NPS.
- **Risk trend (last weeks)** — a line chart of the customer's historical risk scores, so you can see whether the account is recovering or sliding.
- **Log activity** → **re-score loop** — the interactive demo: use the **Log activity** form to record an event (e.g. a login, a support ticket, a failed payment), then re-score to watch the risk score and band shift in response. It shows the pipeline is live, not static.
- **Activity timeline** — the raw `ActivityEvent` stream for the account, newest first.

4 • The AI Risk Scorer

The headline feature: turn a raw stream of activity events into an explainable 0–100 churn score, a band, a written rationale and a recommended action — with a scorer that **always** returns an answer.

How the AI works — explainable hybrid scoring. ChurnGuard is **not** a trained ML model. It is **LLM-as-judge over deterministic features**, with a rule-based fallback (all in `src/lib/scoring.ts`):

- **(a) extractFeatures()** — turns the raw `ActivityEvent[]` into churn signals: `daysSinceSignup`, `daysSinceLastLogin`, `logins30d`, `featureUses30d`, `supportTickets30d`, `failedPayments30d`, `latestNps`, `mrr`, `plan`. Fully deterministic — the same events always produce the same features.
- **(b) scoreCustomer()** — a cascade returning the first tier that succeeds:
 - **1 • Claude** — the Anthropic SDK with **forced tool-use** (`report_churn_risk`, an `input_schema` of `{score, band, reason, action}`), so the model must return structured output — no JSON parsing. Model `claude-sonnet-5`.
 - **2 • Gemini** — a raw REST call to `gemini-flash-latest` using a `responseSchema` for structured JSON. A free-tier fallback with no extra dependency.
 - **3 • Heuristic** — a transparent additive rule engine that always works with no API key: never-logged-in `+40`, no login > 30 days `+35`, failed payments `×15` (cap 30), support tickets `×6` (cap 18), NPS ≤ 6 `+15`. Band: score ≥ 66 → **HIGH**, ≥ 33 → **MEDIUM**, else **LOW**.

Both LLM tiers receive one **byte-for-byte identical prompt**. `scoreAndSave()` runs the scorer, then inside a `prisma.$transaction` **upserts** the current `RiskScore` and **appends** a `RiskScoreHistory` snapshot (which powers the trend chart). The `RiskScore.model` field records which engine produced the score — that is the "Scored by" line on the detail page.

Because the score, the band thresholds and the features are all deterministic, the LLM's job is narrow and explainable: it writes the **reason** and the **action**. If no key is set, the heuristic produces the same shape of result — so the product never breaks.

5 • Likely interview questions

Q: Is the AI making up the score?

No. The churn features and the band thresholds (≥ 66 HIGH, ≥ 33 MEDIUM) are computed deterministically from the event stream. The LLM acts as a judge that writes the *reason* and the *recommended action*; and if no LLM is available, the transparent heuristic produces the same shape of output. It is explainable, not a black box.

Q: Why LLM-as-judge instead of a trained ML churn model?

No labelled training set is needed, it explains itself in plain English, and it degrades gracefully — the deterministic features plus the heuristic fallback mean the app always returns a defensible score even with zero API keys. A trained model would be a black box that needs data I don't have.

Q: How do you get structured JSON out of an LLM?

Forced tool-use, not text parsing. For Claude the output shape is the `report_churn_risk` tool's `input_schema` and `tool_choice` forces the model to call it; for Gemini I pass a `responseSchema`. I read typed `{score, band, reason, action}`, never free text.

Q: How does the live re-score demo work?

The Log-activity form POSTs a new `ActivityEvent`; re-scoring re-runs `extractFeatures()` over the now-updated event stream, calls the scorer cascade, and `scoreAndSave()` upserts the new `RiskScore` and appends a history row — so the score, band and trend chart all update live.

Q: Why keep a score history instead of just the current score?

Trajectory matters more than a snapshot in customer success. Appending a `RiskScoreHistory` row on every re-score lets the trend chart show whether an account is recovering or sliding, which drives the intervention decision.

Q: How does the event-ingest path map to the real world?

The `customers/[id]/events` API route is the webhook path: in production your product would POST usage events (logins, feature uses, tickets, failed payments) into it, and a scheduled re-score would keep the risk book current — the "Log activity" form is just a manual stand-in for that webhook.